

Day 13–14: Inheritance

- Types of inheritance
- extends keyword
- Method overriding
- super keyword

Programs:

- Animal hierarchy
- Vehicle system

Chapter Overview

Ippudu varaku manam nerchukunna topics:

- Variables
- Methods
- Classes
- Objects
- Constructors

Ippudu oka very important OOP concept loki enter avutunnam.

Inheritance

Idi Java lo heart of OOP.

Almost every Java project lo inheritance untundi.

First Story

Imagine...

School lo Principal unnaru.

Principal ki properties:

- name
- age

- address

Principal ki oka son unnadu.

Son ki malli

- name
- age
- address

untayi.

Question:

Son class lo

name

age

address

malli malli rayala?

No.

Already father daggara unnayi kada.

Son automatically those properties pondutadu.

Real life lo idi inheritance.

Programming lo kuda same.

Suppose

Animal

class undi.

Animal ki

eat()

sleep()

methods unnayi.

Dog kuda animal.

Cat kuda animal.

Lion kuda animal.

Prathi class lo

eat()

sleep()

malli rayadam waste.

Instead

Dog Animal nundi inherit cheskuntundi.

Definition

Inheritance ante

Existing class yokka properties and methods ni new class use chesukovadam.

Simple ga

Parent

↓

Child

Parent lo unna features child ki vastayi.

Why Inheritance?

Suppose inheritance lekapothe

Dog

eat()

sleep()

walk()

Cat

eat()

sleep()

jump()

Lion

eat()

sleep()

hunt()

Observe

eat()

sleep()

3 classes lo same.

Waste.

Instead

Animal

eat()

sleep()

Dog

walk()

Cat

jump()

Lion

hunt()

Code small.

Easy.

Reusable.

Advantages

1 Reusability

Already rasina code malli use cheyochu.

2 Less Code

Duplicate code undadu.

3 Easy Maintenance

eat() change cheyyali.

Animal class lo okasari change chesthe enough.

4 Faster Development

Already unna classes ni use cheskovachu.

5 Real World Representation

Real world relationships easy ga represent chestundi.

Parent Class

Parent class ante

Original class.

Example

Animal

Child Class

Parent ni inherit chesina class.

Example

Dog

Diagram

Animal

↑

Dog

Dog Animal ni inherit chesindi.

Java lo inheritance ela chestham?

Using

extends

keyword.

Syntax

```
class Parent
```

```
{
```

```
}
```

```
class Child extends Parent
```

```
{
```

```
}
```

Meaning

Child class Parent class ni inherit chestundi.

Example

```
class Animal
```

```
{
```

```
    void eat()
```

```
    {
```

```
        System.out.println("Animal eats");
```

```
    }
```

```
}
```

```
class Dog extends Animal
```

```
{
```

```
}
```

Dog class lo eat() rayaledu.

Kani use cheyochu.

Main

```
public class Main
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        Dog d = new Dog();
```

```
        d.eat();
```

```
    }
```

```
}
```

Output

Animal eats

Question

Dog class lo eat() ledu.

Mari ela work ayindi?

Answer

Because

Dog extends Animal

Java first Dog lo search chestundi.

Kanipinchakapothe

Parent class Animal lo search chestundi.

Akkada dorikindi.

Execute chestundi.

Memory Diagram

Animal

eat()

sleep()

↓

Dog

walk()

Dog object

eat()

sleep()

walk()

anni access chestundi.

Parent object

Animal a = new Animal();

Can call

eat()

sleep()

Only.

Child object

Dog d = new Dog();

Can call

eat()

sleep()

walk()

Child parent ni access chestundi.

Question

Parent child ni access chestunda?

No.

Example

Animal

↓

Dog

Dog ki

bark()

method undi.

Animal ki teliyadu.

So

Animal a = new Animal();

a.bark();

Error.

Reason

Parent ki child future lo em create chestundo teliyadu.

Types of Inheritance

Java supports different inheritance relationships.

1 Single Inheritance

One parent

One child

Animal

↓

Dog

Example

```
class Animal
```

```
{
```

```
}
```

```
class Dog extends Animal
```

```
{
```

```
}
```

2 Multilevel Inheritance

Animal

↓

Dog



Puppy

Puppy ki

Animal properties

-

Dog properties

vastayi.

Example

```
class Animal
```

```
{  
    void eat()  
    {  
        System.out.println("Eat");  
    }  
}
```

```
class Dog extends Animal
```

```
{  
    void bark()  
    {  
        System.out.println("Bark");  
    }  
}
```

```
class Puppy extends Dog
{
    void play()
    {
        System.out.println("Play");
    }
}
```

Main

```
Puppy p = new Puppy();
```

```
p.eat();
```

```
p.bark();
```

```
p.play();
```

Output

Eat

Bark

Play

Memory

Animal

↓

Dog

↓

Puppy

Puppy object

eat()

bark()

play()

3 Hierarchical Inheritance

One parent

Multiple children.

Animal

/ | \

Dog Cat Lion

Animal methods

↓

Dog

↓

Cat

↓

Lion

Andariki common.

Example

class Animal

{

void eat()

{

System.out.println("Eat");

```
}  
}
```

```
class Dog extends Animal  
{  
  
}
```

```
class Cat extends Animal  
{  
  
}
```

Dog

eat()

Cat

eat()

Both use cheyochu.

4 Multiple Inheritance

A

\

\

C

/

B

Java lo **classes** tho **NOT supported**.

Example

Father

↓

Child

Mother

↓

Child

Problem.

Father

walk()

Mother

walk()

Child

walk()

Which walk()?

Confusion.

Idi **Diamond Problem** ani antaru.

Kabatti Java classes lo multiple inheritance allow cheyyadu.

Note: Java **interfaces** dwara multiple inheritance support chestundi (idi manam later chapters lo detail ga nerchukuntam).

5 Hybrid Inheritance

Multiple + Hierarchical mix.

Java classes tho support cheyyadu.

Interfaces tho possible.

extends Keyword

Syntax

```
class Dog extends Animal
```

```
{
```

```
}
```

Meaning

Dog

↓

Animal ni inherit chestundi.

What comes from Parent?

Child gets

☑ Variables

☑ Methods

✗ Constructors (constructors are **not inherited**; parent constructor runs during child object creation, but the child does not own it)

Example

```
class Animal
```

```
{
```

```
    int age = 10;
```

```
    void eat()
```

```
{
```

```
        System.out.println("Eat");
```



```
}  
}
```

```
class Dog extends Animal  
{  
  
}
```

Main

```
Dog d = new Dog();
```

```
System.out.println(d.age);
```

```
d.eat();
```

Output

10

Eat

Method Overriding

Most important interview topic.

Suppose

Animal

```
void sound()
```

```
{  
    System.out.println("Animal sound");  
}
```

Dog

```
void sound()
```

```
{  
    System.out.println("Bark");  
}
```

Same method.

Different implementation.

Idi

Method Overriding.

Definition

Parent method ni

Child class lo

same name

same parameters

same return type (or a compatible covariant return type for object returns)

tho malli define cheyyadam.

Example

class Animal

```
{  
    void sound()  
    {  
        System.out.println("Animal Sound");  
    }  
}
```

class Dog extends Animal

```
{  
    void sound()  
    {
```

```
        System.out.println("Bark");
    }
}
```

Main

```
Dog d = new Dog();
```

```
d.sound();
```

Output

Bark

Animal sound execute avvaledu.

Child method execute ayindi.

Why Overriding?

Animal

```
sound()
```

Dog

Bark

Cat

Meow

Lion

Roar

Prathi animal sound different.

Kabatti override chestam.

Rules of Overriding

Rule 1

Method name same.

```
sound()
```

Rule 2

Parameters same.

Rule 3

Return type same or compatible.

Rule 4

Child method access level should not be more restrictive than parent.

Example

Parent

```
public void show()
```

Child

```
private void show()
```

Not allowed.

@Override Annotation

Best practice.

```
@Override
```

```
void sound()
```

```
{
```

```
}
```

Compiler check chestundi.

Spelling mistake unte immediate error chupisthundi.

super Keyword

Another important topic.

this gurinchi manam already chusam.

this

Current object.

super

Parent object reference.

Simple.

this → Current class

super → Parent class

Why super?

Three major uses.

1 Parent Variable Access

Example

```
class Animal
```

```
{  
    int age = 10;  
}
```

```
class Dog extends Animal
```

```
{  
    int age = 5;  
  
    void display()  
    {  
        System.out.println(super.age);  
  
        System.out.println(this.age);  
    }  
}
```

Output

10

5

Memory

Parent age =10

Child age =5

super

↓

Parent

this

↓

Child

2 Parent Method Call

Example

class Animal

```
{  
    void sound()  
    {  
        System.out.println("Animal");  
    }  
}
```

class Dog extends Animal

```
{  
    void sound()
```

```
{  
    System.out.println("Dog");  
}
```

```
void display()  
{  
    super.sound();
```

```
    sound();  
}
```

```
}
```

Output

Animal

Dog

Without super

Only

Dog

print avthundi.

3 Parent Constructor Call

Example

class Animal

```
{  
    Animal()  
    {  
        System.out.println("Animal Constructor");  
    }  
}
```

```
}
```

```
class Dog extends Animal
```

```
{
```

```
    Dog()
```

```
    {
```

```
        System.out.println("Dog Constructor");
```

```
    }
```

```
}
```

```
Main
```

```
Dog d = new Dog();
```

```
Output
```

```
Animal Constructor
```

```
Dog Constructor
```

```
Question
```

```
super() rayaledu.
```

```
Mari Animal constructor ela call ayindi?
```

```
Answer
```

```
Compiler automatically first line lo
```

```
super();
```

```
insert chestundi.
```

```
So internally
```

```
Dog()
```

```
{
```

```
    super();
```

```
    System.out.println("Dog Constructor");
```



```
}
```

this vs super

this	super
Current class	Parent class
Current variables	Parent variables
Current methods	Parent methods
Current constructor	Parent constructor

Program 1: Animal Hierarchy

```
class Animal
{
    void eat()
    {
        System.out.println("Animal is eating");
    }
}
```

```
class Dog extends Animal
{
    void bark()
    {
        System.out.println("Dog is barking");
    }
}
```

```
class Puppy extends Dog
{
```

```
void play()
{
    System.out.println("Puppy is playing");
}
}
```

```
public class Main
{
    public static void main(String[] args)
    {
        Puppy p = new Puppy();

        p.eat();
        p.bark();
        p.play();
    }
}
```

Output

Animal is eating

Dog is barking

Puppy is playing

Program 2: Vehicle System

```
class Vehicle
{
    void start()
    {
        System.out.println("Vehicle starts");
    }
}
```

```
}
```

```
class Car extends Vehicle
```

```
{
```

```
    @Override
```

```
    void start()
```

```
    {
```

```
        System.out.println("Car starts with key");
```

```
    }
```

```
}
```

```
class Bike extends Vehicle
```

```
{
```

```
    @Override
```

```
    void start()
```

```
    {
```

```
        System.out.println("Bike starts with self");
```

```
    }
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Car c = new Car();
```

```
        Bike b = new Bike();
```

```
        c.start();
```

```
        b.start();
```

```
}  
}
```

Output

Car starts with key

Bike starts with self

Common Interview Questions

1. What is Inheritance?

Existing class properties and methods ni new class reuse cheyyadam.

2. Which keyword is used for inheritance?

extends

3. Does Java support multiple inheritance with classes?

No. Diamond Problem valla support cheyyadu.

4. What is method overriding?

Parent method ni child class lo same signature tho redefine cheyyadam.

5. Difference between overloading and overriding?

Overloading

Same class lo kuda cheyochu

Method name same, parameters different

Compile-time polymorphism

Overriding

Parent-Child relation compulsory

Method name & parameters same

Runtime polymorphism

6. What is super?

Parent class members ni access cheyadaniki use ayye keyword.

7. What is this?

Current object ni refer chestundi.

8. Are constructors inherited?

No. Constructors **inherit kaavu**, but child object create chestappudu parent constructor first execute avthundi (usually through an implicit super() call).

Easy Memory Trick

Inheritance

↓

Code reuse

extends

↓

Parent → Child connection

Overriding

↓

Same method

New behavior

super

↓

Parent access

this

↓

Current object access

Quick Revision

- **Inheritance** = Existing class features ni new class reuse cheyyadam.
- **Parent class** = Original class.
- **Child class** = Parent ni inherit chesina class.
- **extends** = Inheritance create cheyadaniki use chestam.
- **Single, Multilevel, Hierarchical** inheritance Java classes lo support chestundi.
- **Multiple, Hybrid** inheritance Java classes lo support cheyyadu (interfaces tho possible).
- **Method Overriding** = Parent method ni child class lo same signature tho redefine cheyyadam.
- **super** = Parent variable, method, constructor ni access cheyadaniki.
- **this** = Current class object ni refer cheyadaniki.

- **Constructors inherit kaavu**, but child constructor start ayye mundu parent constructor execute avthundi.
- **Child object** parent members ni use cheyochu; **parent object** child-specific members ni direct ga use cheyyaledu.